

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – Konsep Dasar Pemrograman Berorientasi Objek



Farrel Labib Alhaidar

103132400009

S1SD-05-A

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pemrograman Berorientasi Objek menyusun program ke dalam unit komponen mandiri melalui empat pilar utama yaitu:

- abstraksi untuk menyaring ciri dan perilaku esensial objek.
- enkapsulasi untuk memaketkan data internal bersama *method*-nya guna melindungi proses (*data hiding*) serta mendukung modularitas.
- pewarisan (*inheritance*) untuk menurunkan dan memperluas sifat kelas induk (*superclass*) ke kelas anak (*subclass*).
- polimorfisme yang memungkinkan satu antarmuka menghasilkan berbagai perilaku berbeda, dengan interaksi antar-objek yang berjalan lewat pengiriman pesan atau pemanggilan *method*.

Kelas berfungsi sebagai rancangan cetak biru (*blueprint*), sedangkan objek merupakan hasil instansiasi fisik dari cetak biru tersebut. Data di dalam kelas disimpan pada dua tipe atribut (*field*), yaitu *class field* yang menggunakan modifier static (terikat langsung ke kelas dan nilainya digunakan bersama oleh seluruh objek) serta *instance field* tanpa modifier static (nilainya berdiri sendiri dan unik bagi setiap instans objek).

Aksi objek diatur melalui *method* yang terbagi menjadi fungsi (memiliki nilai kembalian/*return value*) atau prosedur (bertipe void). Java menerapkan dua mekanisme pengiriman argumen ke dalam *method*: *pass by value* untuk tipe primitif (hanya menyalin nilai memori sehingga variabel asal tidak berubah) dan *pass by reference* untuk objek atau *array*.

B. Guided

1. Kucing.java

```
package guided;

public class Kucing {

    String nama;

    String ras;

    void bersuara() {

        System.out.println(nama + " " + ras + " meow!");
    }
}
```

```
}  
}
```

MainKucing.java

```
package guided;  
  
public class Main {  
    public static void main(String[] args) {  
        Kucing labib = new Kucing();  
        labib.nama = "Yibaa";  
        labib.ras = "Munchkin";  
        labib.bersuara();  
    }  
}
```

Output :

```
-dasar\project\target\classes" guided.Main "  
Yibaa Munchkin meow!
```

2. Rekening.java

```
package guided;  
  
public class Rekening {  
    private int saldo = 0;  
  
    public void tambahSaldo(int jumlah) {  
        saldo = saldo + jumlah;  
        System.out.println("Saldo berhasil ditambahkan!");  
    }  
  
    public void tampilkanSaldo() {  
        System.out.println("Saldo Anda adalah: " + saldo);  
    }  
}
```

MainRekening.java

```
package guided;

public class MainRekening {
    public static void main(String[] args) {
        Rekening rek = new Rekening();
        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}
```

Output :

```
-dasar\project\target\classes" guided.MainRekening "
Saldo berhasil ditambahkan!
Saldo Anda adalah: 20000000
```

3. Circle.java

```
package guided;

public class Circle {
    public static final double PI = 3.14159;

    public static double radiansToDegrees(double rads) {
        return rads * 180 / PI;
    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }
}
```

CircleMain.java

```
package guided;

public class MainCircle {
    public static void main(String[] args) {
        Circle c = new Circle();
        c.r = 7.0;
        System.out.printf("Jari-jari      : %.1f%n", c.r);
        System.out.printf("Luas          : %.5f%n", c.area());
        System.out.printf("Keliling      : %.5f%n",
c.circumference());
        System.out.printf("PI (class field) : %s%n", Circle.PI);
    }
}
```

Output:

```
-dasar\project\target\classes" guided.MainCircle "
Jari-jari      : 7.0
Luas           : 153.93791
Keliling       : 43.98226
PI (class field) : 3.14159
```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

PengolahSuhu.java

```
package unguided;

public class PengolahSuhu {

    private static final double NILAI_KOSONG = -1.0;

    private double[] suhuHarian;

    public PengolahSuhu(double[] suhuHarian) {

        this.suhuHarian = suhuHarian;

    }

    public void tampilkanData() {

        for (int i = 0; i < suhuHarian.length; i++) {

            if (suhuHarian[i] == NILAI_KOSONG) {

                System.out.println("Hari " + (i + 1) + " : (kosong)");

            } else {

                System.out.printf("Hari %d : %.1f\u00B0C\n", (i + 1),
suhuHarian[i]);

            }

        }

    }

    public int cariIndexKosong() {

        for (int i = 0; i < suhuHarian.length; i++) {

            if (suhuHarian[i] == NILAI_KOSONG) {

                return i;

            }

        }

    }

}
```

```

    }

    }

    return -1;

}

public void isiDataKosong() {

    int index = cariIndexKosong();

    if (index != -1) {

        suhuHarian[index] = (suhuHarian[index - 1] +
suhuHarian[index + 1]) / 2;

    }

}

public double hitungRataRata() {

    double total = 0;

    for (double suhu : suhuHarian) {

        total += suhu;

    }

    return total / suhuHarian.length;

}

}

```

Main.java

```

package unguided;

public class Main {

    public static void main(String[] args) {

        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8,
32.9 };

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);
    }

}

```

```

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        int indexKosong = pengolah.cariIndexKosong();
        System.out.println("\nIndex hari kosong (dimulai dari 0): " +
indexKosong);

        pengolah.isiDataKosong();

        System.out.println("\n=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.printf("%nRata-rata : %.2f\u00B0C%n",
pengolah.hitungRataRata());
        System.out.println("\nIsi array suhuHarian di main setelah
isiDataKosong() dijalankan:");
        System.out.print("[");
        for (int i = 0; i < suhuHarian.length; i++) {
            System.out.printf("%.1f", suhuHarian[i]);
            if (i < suhuHarian.length - 1)
                System.out.print(", ");
        }
        System.out.println("]");
        System.out.println("(ikut berubah: constructor menyimpan
referensi array yang sama)");
    }
}

```

Output :

```
ct\target\classes" unguided.Main "  
=== Data Suhu Awal ===  
Hari 1 : 30.4°C  
Hari 2 : 24.3°C  
Hari 3 : 26.8°C  
Hari 4 : (kosong)  
Hari 5 : 31.4°C  
Hari 6 : 30.8°C  
Hari 7 : 32.9°C  
  
Index hari kosong (dimulai dari 0): 3  
  
=== Data Suhu Setelah Pengisian ===  
Hari 1 : 30.4°C  
Hari 2 : 24.3°C  
Hari 3 : 26.8°C  
Hari 4 : 29.1°C  
Hari 5 : 31.4°C  
Hari 6 : 30.8°C  
Hari 7 : 32.9°C  
  
Rata-rata : 29.39°C  
  
Isi array suhuHarian di main setelah isiDataKosong() dijalankan:  
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]  
(ikut berubah: constructor menyimpan referensi array yang sama)
```

Penjelasan :

Program ini mengimplementasikan *class PengolahSuhu* untuk membersihkan dan menganalisis data suhu harian selama tujuh hari menggunakan prinsip OOP. Class tersebut menerapkan enkapsulasi dengan seluruh field bersifat private, memiliki *instance field suhuHarian* yang menyimpan array suhu tiap object, serta *class field NILAI_KOSONG* bertipe static final double bernilai -1.0 sebagai penanda data hilang yang bersifat tetap dan berlaku untuk semua object. Program mampu mendeteksi hari yang datanya kosong, mengisinya menggunakan metode imputasi linear (rata-rata hari sebelum dan sesudah), lalu menghitung rata-rata suhu akhir.

D. Kesimpulan

1. Paradigma PBO mengintegrasikan empat konsep fundamental abstraksi, enkapsulasi, pewarisan (*inheritance*), dan polimorfisme untuk menstrukturkan program, di mana kelas berperan sebagai *blueprint* dan objek merupakan wujud nyata hasil instansiasi dari rancangan tersebut.

2. Distribusi Status Data (*Field*) dan Perilaku (*Method*): Data di dalam kelas dikelola melalui *class field* ber-modifier *static* yang nilainya dibagikan bersama ke seluruh komponen kelas, serta *instance field* yang menyimpan nilai unik untuk tiap instans objek, sementara perilakunya dijalankan melalui *method* berupa fungsi atau prosedur bertipe *void*.
3. Pengolahan data pada *method* terbagi secara tegas antara manipulasi salinan nilai pada tipe primitif (*pass by value*) dan manipulasi langsung pada memori objek/array asli (*pass by reference*), dengan kata kunci *this* yang berfungsi mengarahkan referensi ke objek aktif sekaligus menangani pemanggilan konstruktor berantai (*constructor chaining*).